# Project: Authentication & API Gateway

Due: 03/21/2026 11:59 PM
Extended Deadline: 03/22/2026 11:59 PM



**Introduction**
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

**Ingress + Gateway**
- Task Overview
- Task

**KrakenD**
- Task Overview
- Task

**Django Authentication**
- Task Overview
- Task

**Securing LLM API**
- Task Overview
- Task

**Wrap Up**
- Conclusion

## Django Authentication (Task)

With KrakenD configured, we can now move on to the next task: building part of an API authentication system.

In this task, we will use **Django** another Python web framework to implement a simple login service.

Django is a high-level Python framework designed to enable rapid development of complex, database-driven applications. It follows a “batteries-included” philosophy, providing built-in features such as an ORM, admin interface, authentication system, and templating engine out of the box.

Compared to FastAPI, Django offers a more complete, full-stack solution that is well suited for traditional web applications requiring substantial backend logic and quick prototyping. However, it generally lags behind FastAPI in terms of performance and concurrency due to its historically synchronous core (although async views have been supported since version 3.1).

The goal of this task is to develop a login service. Login services are often among the first microservices you will encounter that rely on persistent state stored in a database. In other coursework, you may have interacted with databases directly using SQL. While SQL is powerful, it requires careful management and optimization. Since that is not our focus here, we will instead rely on Django’s built-in ORM. Django models allow us to interact with the database using Python code rather than writing raw SQL queries.

One of Django’s greatest strengths is its rich ecosystem of built-in and third-party libraries, which make it easy to prototype functionality quickly. In this task, we will use **Django REST Framework (DRF)** and **djangorestframework-simplejwt** to simplify API development.

**Django REST Framework (DRF):** A powerful toolkit for building RESTful APIs in Django. It provides serializers, views, and authentication mechanisms that significantly streamline API development.
**djangorestframework-simplejwt:** An extension for DRF that provides JSON Web Token (JWT) authentication, enabling secure and stateless API access.

**Information**

While we aim to move through Django efficiently, it is important to note that this section will be challenging. We expect this to be one of the more difficult tasks, so please do not hesitate to ask questions.

Additionally, keep in mind that Django REST Framework introduces significant changes to traditional Django workflows. Since this course focuses on backend and cloud-native development rather than frontend design, we believe this tradeoff is appropriate.

## Subtask 1: Getting Started with Django

To begin working with Django, we will start by creating a new Poetry project and then use `django-admin` to scaffold a Django application. Follow the steps below carefully.

```
# From the project root, create a new direc
mkdir django-authentication

# Move into the directory
cd django-authentication/

# Initialize a new Poetry project
poetry init -n
```

Inside the `django-authentication` directory, create a file named `poetry.toml` and add the following configuration:

```
[virtualenvs]
in-project = true
path = ".venv"
```

As you may remember from the previous project, this configuration tells Poetry to store the virtual environment

inside the project directory itself. This keeps everything self-contained and ensures that deleting the project also removes the virtual environment.

Next, install the required dependencies:

```
# Install dependencies (not all will be us
poetry add django djangorestframework djan

# Use django-admin to create a new Django
poetry run django-admin startproject task3
```

This creates a new Django project. By default, Django is designed around a monolithic architecture, where the application is structured as a single cohesive unit. However, Django also supports modular development through *apps*, which are self-contained components within a project. You can think of these apps as smaller services inside a larger monolith.

For this task, we will keep things simple by creating a single app.

```
cd task3

# Create a Django app named "login"
poetry run django-admin startapp login
```

After running these commands, your directory structure should look similar to the following:

```
.
├── README.md
├── answers
├── django-authentication
│   ├── manage.py
│   ├── poetry.lock
│   ├── poetry.toml
│   ├── pyproject.toml
│   └── task3
│       ├── __init__.py
│       ├── asgi.py
│       ├── login
│       │   ├── __init__.py
│       │   ├── admin.py
│       │   ├── apps.py
│       │   ├── models.py
```

```
│       │   ├── tests.py
│       │   └── views.py
│       ├── settings.py
│       ├── urls.py
│       └── wsgi.py
├── k8s-componenets/
├── llm_server/
├── ruff.toml
├── submitter
├── terraform
├── terraform_init.log
└── tree.txt
```

Django enforces a structured and opinionated project layout. While this makes large applications easier to maintain, it can feel restrictive at first, as multiple files must be configured correctly for even simple functionality. Unlike minimal frameworks where a single file may be sufficient, Django requires coordination across several components.

Before moving forward, we need to run a few essential setup commands.

Make sure you're in the `django-authentication` directory, then run the database migrations:

```
poetry run python manage.py migrate
```

Although `django-admin` is used to create projects and apps, most day-to-day development tasks are performed using `manage.py`. This command initializes the database and creates a `db.sqlite3` file in the project directory.

By default, Django uses SQLite, a lightweight file-based database. This is sufficient for development and learning purposes, and we will use it for now before discussing production-grade databases and scaling strategies later.

Next, create a superuser account:

```
poetry run python manage.py createsuperuse
```

When prompted, set the password to `asdasd` (specifically) and type `y` to bypass validation.

This superuser account gives you full administrative access to the Django application.

Now, start the development server:

```
poetry run python manage.py runserver
```

**Information**

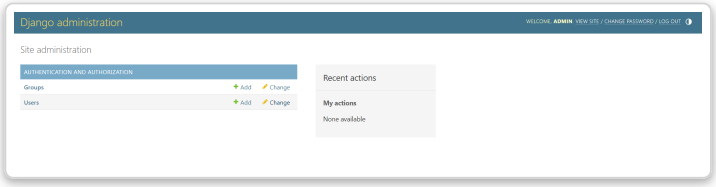
If you're developing using an Azure VM, you have to expose port 8000 from the VM network configuration settings on Azure.
Once it's exposed your can access the Django app using `http://YOUR_VM_IP:8000/admin/login/`

Then, go to the task3 directory, open the `settings.py` file, and set ALLOWED_HOSTS = ["*"].
For development purposes, we'll be opening our app to the entire internet. In production, however, you shouldn't do this. Only authorized hosts should be given access to your application.

Navigate to the following URL in your browser:

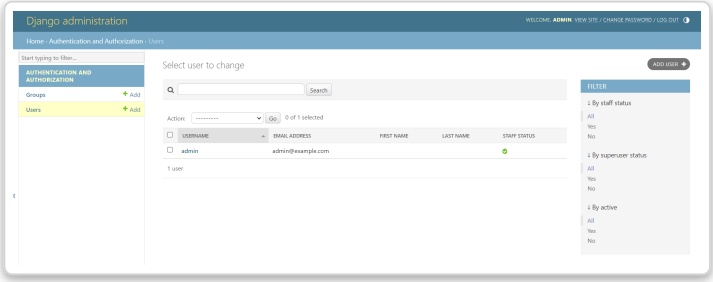
```
http://localhost:8000/admin/login/
```

You should see the Django admin login page. Log in using the credentials you just created. Once logged in, you will be taken to the admin dashboard.




From this interface, you can manage users, groups, and other application data without writing any additional code. Django provides this functionality out of the box because user management and database interaction are fundamental requirements for most applications.

If you click the green “+” icon next to **Users**, you can create additional users. Clicking on **Users** itself will show a list of all existing users in the system.




Now that we've explored the Django admin interface, we are ready to move on to the next step: building APIs using **Django REST Framework**, which will form the core of our authentication service going forward.

## Subtask 2: Django REST Framework

Django REST Framework (DRF) is a powerful extension of Django that makes it easy to build RESTful APIs. In this section, we'll walk through creating a simple API endpoint that returns a list of users in our application.

This endpoint will return all users stored in the system and will only be accessible to authenticated users. Since Django already provides built-in user management, we don't need to create a custom database table—Django has already created one for us. The admin user we created earlier is stored in this table as well. Our task is simply to retrieve this data and expose it through an API.

Django and Django REST Framework separate responsibilities cleanly across different components such as routing, request handling, data modeling, and serialization. This separation makes applications easier to maintain and reason about.

Here's how Django structures the workflow for this task:

> **Routing**: Defined in `task3/urls.py`, where we specify which URL maps to which view.
> **Views**: Defined in `task3/login/views.py`, where request handling logic lives.
> **Models**: Defined in `task3/login/models.py`, where database schemas are defined. For this task, we'll use Django's built-in `User` model.
> **Serializers (DRF-specific)**: Defined in `task3/login/serializers.py`, responsible for converting model data into JSON and vice versa.

To implement this feature, we will follow three steps:

1. **Define the Serializer** – Converts `User` model instances into JSON.

2. **Create the View** – Fetches users from the database and returns them using the serializer.
3. **Add the URL Route** – Exposes the endpoint through Django's routing system.

---

## Step 1: Define the Serializer

Create a file named `serializers.py` inside the `task3/login` directory and add the following code:

```
from django.contrib.auth.models import Gro
from rest_framework.serializers import Mod

class UserSerializer(ModelSerializer):
    class Meta:
        model = TODO1
        fields = TODO2
```

Although the syntax may look unfamiliar at first, the key idea is simple:

> **ModelSerializer** automatically converts a Django model into a JSON-friendly format.
> **model** specifies which database table to use. In our case, this will be the built-in `User` model.
> **fields** defines which columns from the table should be included in the API response. For this task, we only want the username and email.

Django abstracts away raw SQL by allowing you to interact with the database using Python classes. You rarely need to write SQL directly—Django handles that for you behind the scenes.

Complete the serializer by filling in the missing fields before proceeding to the next step.
We strongly encourage you to familiarize yourself with reading the documentation on Django. Here's a link to the documentation on Model Serializers.

## Step 2: Create the Views

Open `task3/login/views.py`. You should see a file that contains only:

```
from django.shortcuts import render
```

The `render` helper is Django's shorthand for returning templated HTML responses in traditional server-rendered

applications. Without DRF, we would need to manually handle request parsing and response formatting. Since we are using DRF and serializers, we can simplify this significantly.

Replace the entire contents of the file with the following:

```
from django.contrib.auth.models import Gro
from rest_framework import permissions, vi
from task3.login.serializers import UserSe

class UserViewSet(viewsets.ModelViewSet):
    """
    API endpoint that allows users to be v
    """
    queryset = TODO
    serializer_class = TODO
    permission_classes = [permissions.IsAu
```

Here's what each part does:

**queryset**: Defines the database query used to retrieve users.
`User.objects` tells Django to query the built-in `User` table, and the rest orders the results by newest users first. Replace the TODO with `User.objects.all().order_by('-date_joined').` Conceptually, this is similar to: `SELECT * FROM User ORDER BY date_joined DESC;`
**serializer_class**: Specifies the serializer used to convert `User` objects into JSON.
We use the `UserSerializer` we created earlier.
**permission_classes**: Restricts access to authenticated users only.
This ensures the endpoint cannot be accessed without valid credentials.

Here's the [documentation on ViewSet]() for your perusal.

Once you've completed the view, the next step is to connect it to URLs.

---

## Step 3: Adding the Routes

Open `task3/urls.py`, which should look like this:

```
from django.contrib import admin
from django.urls import path

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

The `urlpatterns` list defines all routes available in your application. By default, Django includes the `admin/` route, which provides the built-in admin interface.

Next, we will add two additional routes:

- **The DRF authentication routes** (`api-auth/`), used for login/logout in the DRF browsable API.
- **The users endpoint**, which will expose the `UserViewSet` API for listing users.

### Step 3.1: Add the `api-auth` Route

First, import `include` from `django.urls` and update the `urlpatterns` list as shown below.

This enables Django REST Framework's built-in authentication views, which provide a simple login/logout interface for the browsable API.

```
path('api-auth/', include('rest_framework.
```

---

### Step 3.2: Add the Users Route

To expose our users API, we'll use DRF's router system. Routers automatically generate RESTful routes for viewsets, saving us from writing repetitive URL mappings.

Add the following code **below** the `urlpatterns` declaration:

```
from rest_framework import routers
from task3.login import views

router = routers.DefaultRouter()
router.register(r'users', views.UserViewSe
urlpatterns += router.urls
```

Here's what this does:

`routers.DefaultRouter()`
Creates a router that automatically handles REST-style routing.

`router.register(r'users', views.UserViewSet)`
Registers the `/users/` endpoint and maps it to the `UserViewSet` class.

`urlpatterns += router.urls`
Appends all auto-generated routes to Django's main URL configuration.

With this in place, DRF automatically creates endpoints such as:

- `/users/` – list users
- `/users/<id>/` – retrieve a specific user

---

### Step 4: Add Apps to `INSTALLED_APPS`

Django does not automatically detect newly created apps. You must explicitly register them.

Open `task3/settings.py` and add the following to the `INSTALLED_APPS` list:

```
'rest_framework',
```

(Your custom `login` app should already be listed if Django added it automatically.)

This ensures Django loads Django REST Framework correctly.

---

### Step 5: Run the Application

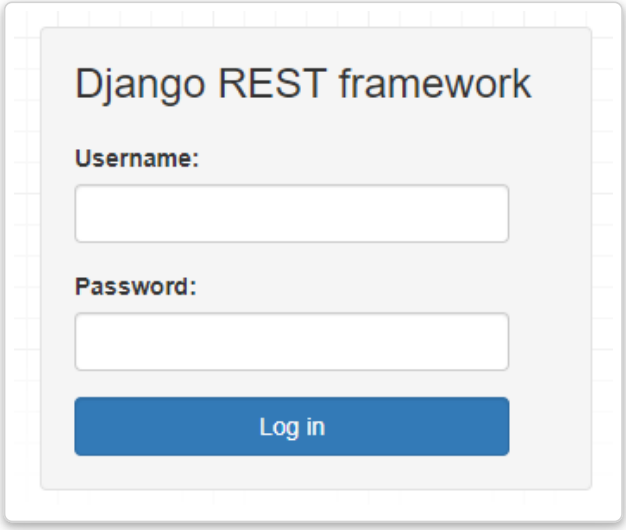
Start the development server:

```
poetry run python manage.py runserver
```

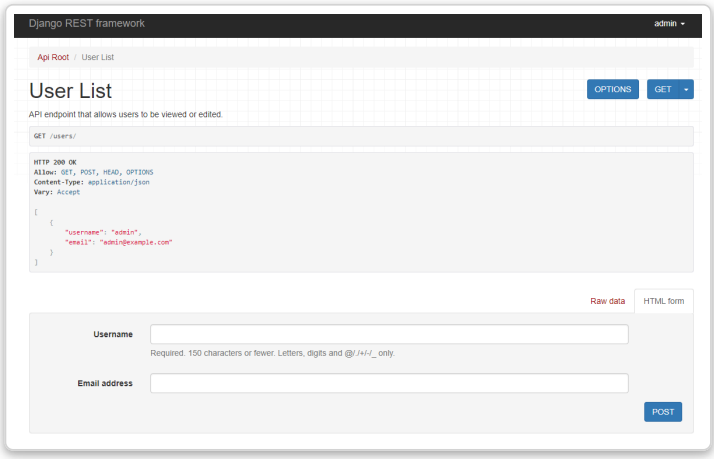
Now open the following URL in your browser:

```
http://localhost:8000/users
```

If you are not logged in, Django will redirect you to the login page. Log in using the superuser credentials you created earlier.




Once authenticated, you should see a JSON response containing all users in the system.




---

**Information**

Django uses a setting called `ALLOWED_HOSTS` (in `settings.py`) to control which hosts are allowed to access the application.

If you try to access the app using an IP or hostname that is not listed, Django will throw a **"Host not allowed"** error.

For this project, you can safely set:

```
ALLOWED_HOSTS = ["*"]
```

In production environments, this should **never** be a wildcard. Instead, you should explicitly list the domain names or IP addresses that are allowed to access your backend.

## Subtask 3: JWT Login

Now that we can list users and log in through the browser, we need a way to authenticate

programmatically—so services and clients can securely call our APIs without relying on an interactive login session. To do this, we will implement a simple REST interface that issues authentication tokens.

A common approach for programmatic authentication is **JSON Web Tokens (JWTs)**. JWTs are a standard mechanism for proving authentication to an API, and they are supported by most API gateways (including KrakenD). They are lightweight, stateless, and easy to attach to each request.

For this task, we will use **djangorestframework-simplejwt** to generate and sign JWTs. Client programs can submit user credentials to receive a token, then include that token in subsequent requests (typically via the `Authorization: Bearer <token>` header).

`djangorestframework-simplejwt` integrates cleanly with Django REST Framework and is quick to set up.

---

## Step 1: Configure DRF to Use JWT Authentication

In `settings.py`, add the following configuration. This tells Django REST Framework to authenticate incoming API requests using JWT tokens via `JWTAuthentication`.

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authenti
    )
}
```

---

## Step 2: Add Token Routes in `urls.py`

Import the SimpleJWT views and add the token endpoints to your `urlpatterns` list:

```
from rest_framework_simplejwt.views import
    TokenObtainPairView,
    TokenRefreshView,
)

urlpatterns = [
    ...
```

```
    path('api/token/', TokenObtainPairView
    path('api/token/refresh/', TokenRefresl
    ...
]
```

What these routes do:

- `TokenObtainPairView` (`/api/token/`)
  Accepts a username and password and returns:
  - an **access token** (short-lived; default is 5 minutes)
  - a **refresh token** (longer-lived; default is 1 day)
- `TokenRefreshView` (`/api/token/refresh/`)
  Accepts a refresh token and returns a new access token when the old access token expires.

---

### Step 3: Add SimpleJWT to `INSTALLED_APPS`

In `settings.py`, add the following entry to your `INSTALLED_APPS` list:

```
'rest_framework_simplejwt',
```

---

### Step 4: Validate the Token Endpoints

Start the server if it is not already running:

```
poetry run python manage.py runserver
```

You can validate the endpoints using `curl`:

```
curl \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{"username":"admin", "password":"asd
  http://localhost:8000/api/token/
```

If successful, the response will include an `access` token and a `refresh` token.

In the next task, we will deploy this authentication service and integrate it with KrakenD.

---

## End of Task Validation

Make sure the server is running before executing the grader. The grader will test your service at:

```
http://127.0.0.1:8000/
```

## Submitting

Django makes development faster by providing robust built-in functionality. Without these libraries, implementing this service in Flask would require significantly more custom code.

Run the following to submit:

```
./submitter -t task3
```

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform